In [1]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

# 设置中文显示（避免 SimHei 未定义）
plt.rcParams['font.sans-serif'] = ['Microsoft YaHei', 'SimHei', 'Arial Unicode M
plt.rcParams['axes.unicode_minus'] = False

# 使图表在 notebook 中显示
%matplotlib inline
```

In [2]:
```
# 请根据实际文件路径修改，这里假设 training_sample.csv 在当前工作目录下
df = pd.read_csv('training_sample.csv', encoding='utf-8')

print(f'数据集大小：{df.shape[0]} 行 × {df.shape[1]} 列')
df.head()
```

```
数据集大小：455401 行 × 25 列
```

Out[2]:

	UserID	basket_icon_click	basket_add_list	basket_add_detail	sort_by	image_
0	a720-6b732349-a720-4862-bd21-644732	0	0	0	0	
1	a0c0-6b73247c-a0c0-4bd9-8baa-797356	0	0	0	0	
2	86a8-6b735c67-86a8-407b-ba24-333055	0	0	0	0	
3	6a3d-6b736346-6a3d-4085-934b-396834	0	0	0	0	
4	b74a-6b737717-b74a-45c3-8c6a-421140	0	1	0	1	

5 rows × 25 columns

In [3]:

```python
# 查看基本信息
df.info()

# 检查缺失值
print(df.isnull().sum())

# 查看目标变量分布
print(df['ordered'].value_counts(normalize=True))
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 455401 entries, 0 to 455400
Data columns (total 25 columns):
 #   Column                   Non-Null Count   Dtype
---  ------                   --------------   -----
 0   UserID                   455401 non-null  object
 1   basket_icon_click        455401 non-null  int64
 2   basket_add_list          455401 non-null  int64
 3   basket_add_detail        455401 non-null  int64
 4   sort_by                  455401 non-null  int64
 5   image_picker             455401 non-null  int64
 6   account_page_click       455401 non-null  int64
 7   promo_banner_click       455401 non-null  int64
 8   detail_wishlist_add      455401 non-null  int64
 9   list_size_dropdown       455401 non-null  int64
 10  closed_minibasket_click  455401 non-null  int64
 11  checked_delivery_detail  455401 non-null  int64
 12  checked_returns_detail   455401 non-null  int64
 13  sign_in                  455401 non-null  int64
 14  saw_checkout             455401 non-null  int64
 15  saw_sizecharts           455401 non-null  int64
 16  saw_delivery             455401 non-null  int64
 17  saw_account_upgrade      455401 non-null  int64
 18  saw_homepage             455401 non-null  int64
 19  device_mobile            455401 non-null  int64
 20  device_computer          455401 non-null  int64
 21  device_tablet            455401 non-null  int64
 22  returning_user           455401 non-null  int64
 23  loc_uk                   455401 non-null  int64
 24  ordered                  455401 non-null  int64
dtypes: int64(24), object(1)
memory usage: 86.9+ MB
UserID                     0
basket_icon_click          0
basket_add_list            0
basket_add_detail          0
sort_by                    0
image_picker               0
account_page_click         0
promo_banner_click         0
detail_wishlist_add        0
list_size_dropdown         0
closed_minibasket_click    0
checked_delivery_detail    0
checked_returns_detail     0
sign_in                    0
saw_checkout               0
saw_sizecharts             0
saw_delivery               0
saw_account_upgrade        0
saw_homepage               0
device_mobile              0
device_computer            0
device_tablet              0
returning_user             0
loc_uk                     0
ordered                    0
dtype: int64
ordered
0    0.958074
```

```
1    0.041926
Name: proportion, dtype: float64
```

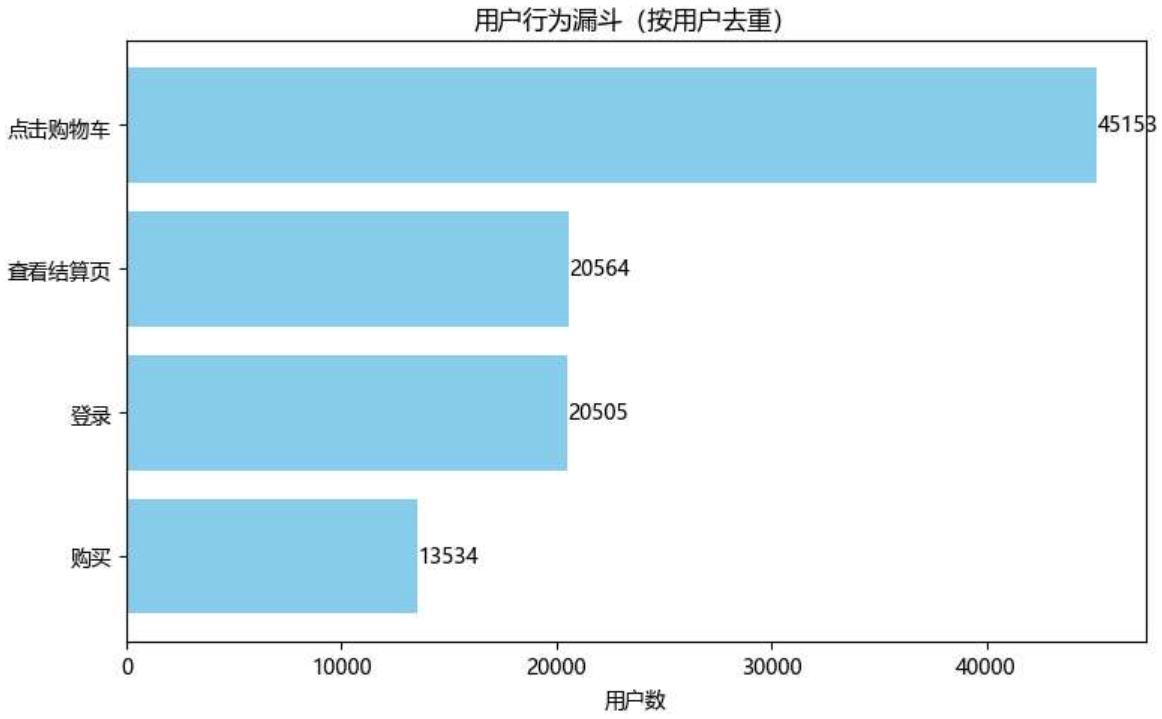
In [4]:

```python
# 定义行为列（按业务顺序）
steps = ['basket_icon_click', 'saw_checkout', 'sign_in', 'ordered']
step_names = ['点击购物车', '查看结算页', '登录', '购买']

# 按用户去重计算每个步骤的用户数
def count_users(condition):
    return df[condition]['UserID'].nunique()   # 注意这里是 nunique()

# 依次计算满足条件的用户数
user_counts = []
current_mask = pd.Series(True, index=df.index)  # 初始全 True
for step in steps:
    current_mask = current_mask & (df[step] == 1)
    user_counts.append(count_users(current_mask))

# 构建漏斗数据框
funnel = pd.DataFrame({
    '阶段': step_names,
    '用户数': user_counts
})
funnel['转化率（相对上一步）'] = (funnel['用户数'] / funnel['用户数'].shift(1) *
funnel.loc[0, '转化率（相对上一步）'] = 100.0
print(funnel)
```

```
     阶段    用户数  转化率（相对上一步）
0  点击购物车  45153      100.00
1  查看结算页  20564       45.54
2     登录  20505       99.71
3     购买  13534       66.00
```

In [5]:

```python
fig, ax = plt.subplots(figsize=(8, 5))
y_pos = range(len(funnel))
ax.barh(y_pos, funnel['用户数'], color='skyblue')
ax.set_yticks(y_pos)
ax.set_yticklabels(funnel['阶段'])
ax.invert_yaxis()
ax.set_xlabel('用户数')
ax.set_title('用户行为漏斗（按用户去重）')
for i, val in enumerate(funnel['用户数']):
    ax.text(val + 10, i, str(val), va='center')
plt.tight_layout()
plt.show()
```

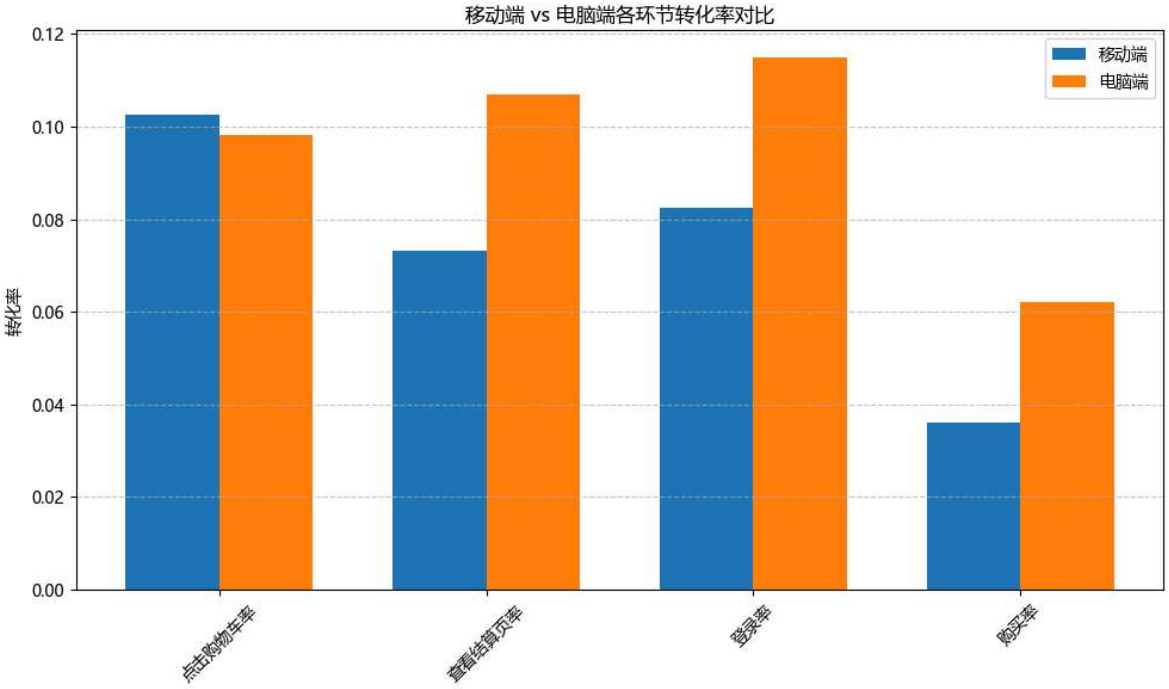


In [6]:
```
# 定义计算转化率的函数（按用户去重）
def device_conversion(device_col):
    device_df = df[df[device_col] == 1]
    total_users = device_df['UserID'].nunique()
    click_cart = device_df[device_df['basket_icon_click'] == 1]['UserID'].nuniqu
    saw_checkout = device_df[device_df['saw_checkout'] == 1]['UserID'].nunique()
    sign_in = device_df[device_df['sign_in'] == 1]['UserID'].nunique()
    ordered = device_df[device_df['ordered'] == 1]['UserID'].nunique()
    return {
        '总用户数': total_users,
        '点击购物车率': round(click_cart / total_users, 4),
        '查看结算页率': round(saw_checkout / total_users, 4),
        '登录率': round(sign_in / total_users, 4),
        '购买率': round(ordered / total_users, 4)
    }

# 计算移动端和电脑端
mobile_stats = device_conversion('device_mobile')
computer_stats = device_conversion('device_computer')

# 转为 DataFrame 并展示
comp_df = pd.DataFrame({
    '移动端': mobile_stats,
    '电脑端': computer_stats
}).T
print(comp_df)
```

```
        总用户数  点击购物车率  查看结算页率    登录率    购买率
移动端  309994.0  0.1025  0.0732  0.0824  0.036
电脑端   88448.0  0.0981  0.1068  0.1150  0.062
```

In [7]:
```
# 提取需要对比的指标
metrics = ['点击购物车率', '查看结算页率', '登录率', '购买率']
mobile_vals = [mobile_stats[m] for m in metrics]
computer_vals = [computer_stats[m] for m in metrics]

x = np.arange(len(metrics))
width = 0.35
```

```
fig, ax = plt.subplots(figsize=(10, 6))
ax.bar(x - width/2, mobile_vals, width, label='移动端')
ax.bar(x + width/2, computer_vals, width, label='电脑端')
ax.set_xticks(x)
ax.set_xticklabels(metrics, rotation=45)
ax.set_ylabel('转化率')
ax.set_title('移动端 vs 电脑端各环节转化率对比')
ax.legend()
ax.grid(axis='y', linestyle='--', alpha=0.7)
plt.tight_layout()
plt.show()
```



分析结论与优化建议

1. 漏斗流失关键点

- 从“点击购物车”到“查看结算页”转化率为 **45.54%**，流失率超过一半，是最大的流失节点。
- “查看结算页”→“登录”转化率为 **99.71%**，说明用户进入结算页后几乎都会登录。
- “登录”→“购买”转化率为 **66.00%**，仍有约三分之一的用户登录后未完成购买，可能存在支付流程或价格因素影响。

2. 登录环节影响显著

登录用户的购买率（66%）远高于整体购买率（4.2%），提升登录率对整体转化帮助很大。

3. 设备端差异

- 电脑端购买率（6.2%）显著高于移动端（3.6%）。
- 移动端在“查看结算页”环节转化率（7.3%）低于电脑端（10.7%），说明移动端用户在查看结算页后更容易跳出。
- 移动端登录率（8.2%）也低于电脑端（11.5%），进一步拉低了购买转化。

优化建议

- **优化结算页引导**：在购物车页面增加醒目的“去结算”按钮，并展示预计送达时间，减少用户犹豫。
- **移动端体验提升**：简化移动端登录流程，支持手机号一键登录或第三方登录（微信/支付宝），降低登录门槛。
- **移动端页面优化**：检查移动端结算页加载速度和表单设计，减少因操作不便导致的流失。
- **针对性运营**：根据设备端表现，向移动端用户推送专属优惠券，提升下单意愿。